

Towards Memory Disaggregation via NVLink C2C: Benchmarking CPU-Requested GPU Memory Access

Felix Werner
HPI, University of Potsdam
Potsdam, Germany
felix.werner@student.hpi.uni-
potsdam.de

Marcel Weisgut
HPI, University of Potsdam
Potsdam, Germany
marcel.weisgut@uni-potsdam.de

Tilmann Rabl
HPI, University of Potsdam
Potsdam, Germany
tilmann.rabl@uni-potsdam.de

Abstract

Memory disaggregation decouples compute and memory resources, enabling efficient use of resources. Several interconnect technologies provide cache-coherent access to remote memory regions, which eases the use of disaggregated memory. Recent NVIDIA-based systems use the NVLink C2C interconnect, which provides cache-coherent memory access between CPUs and GPUs and their memory. While GPUs and NVLink are widely used to accelerate complex workloads, NVLink’s viability for connecting memory-expansion devices to a CPU remains unexplored. In this work, we quantify the characteristics of NVIDIA’s Grace CPU when accessing GPU memory via NVLink to assess NVLink’s viability for memory expansion. We benchmark throughput and latency for memory accesses on an NVIDIA Grace-Hopper system. We evaluate memory expansion when the CPU accesses both CPU and GPU memory and quantify the performance of database index operations with data stored in GPU memory. Our experiments show a throughput of up to 168 GB/s and access latencies between about 800 ns and 1000 ns.

CCS Concepts

• Hardware → Memory and dense storage.

ACM Reference Format:

Felix Werner, Marcel Weisgut, and Tilmann Rabl. 2025. Towards Memory Disaggregation via NVLink C2C: Benchmarking CPU-Requested GPU Memory Access. In *4th Workshop on Heterogeneous Composable and Disaggregated Systems (HCDS ’25)*, March 30, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3723851.3723853>

1 Introduction

Conventional data center architectures suffer from resource underutilization, where the fixed allocation of compute and memory resources results in inefficient resource utilization [21]. Memory disaggregation addresses this challenge by decoupling memory from compute nodes, allowing dynamic and effective use of memory resources [3, 5, 21]. Besides CPU-local memory, memory can be located on different devices, like GPU, FPGA, and ASIC devices. Memory comes in various media types, e.g., dynamic random access memory (DRAM), high bandwidth memory (HBM), or persistent

memory [39]. Memory can be connected through different interconnects, e.g., double data rate (DDR), inter-socket interconnects (e.g., Ultra Path Interconnect (UPI) [41]), Compute Express Link (CXL) [10], NVLink [14, 16], and Infinity Fabric [2].

NVLink [12, 14] supports GPU-to-GPU and, with versions 2 and 4, CPU-to-GPU connections. Its latest version, NVLink 4 (also called NVLink C2C), provides a bidirectional bandwidth of 900 GB/s [14, 16, 17]. The need for memory disaggregation in data centers raises the question of how NVLink-attached memory performs under memory-intensive workloads. No NVLink-connected device exists that is mainly designed for memory expansion. Instead, GPUs and NVLink are widely used to accelerate complex workloads, such as database operators [35–37]. NVLink’s viability for connecting memory located on a device to a CPU purely for memory expansion without utilizing compute resources of the device remains unexplored. While no memory expansion devices with NVLink connectivity exist, NVLink allows cache-coherent access from a CPU to the GPU’s memory. In this work, we quantify the characteristics of NVIDIA’s Grace CPU when accessing GPU memory via NVLink to assess NVLink’s viability for memory expansion and the usage of GPU memory as a far memory tier.

Contributions. Our main contributions are the following:

- (1) We systematically evaluate the throughput and latency of CPU accesses to GPU memory over NVLink C2C and throughput expansion when accessing CPU and GPU memory (Section 3).
- (2) We evaluate the impact of storing data in NVLink C2C-attached GPU memory for database operations by simulating access to a hybrid tree-index structure (Section 4).
- (3) We discuss the suitability of NVLink C2C-attached memory for memory expansion (Section 5).

2 Background

We introduce NVLink, its memory access anatomy, and the benchmark framework used in our evaluation.

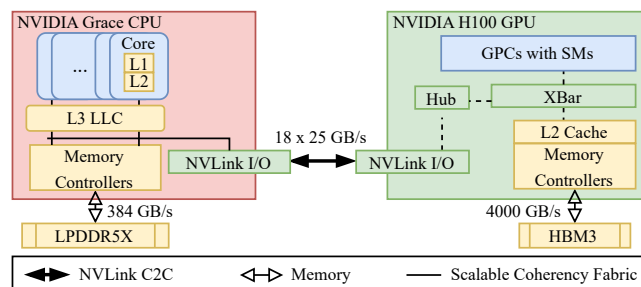


Figure 1: NVIDIA GH200 connecting an NVIDIA Grace CPU with an NVIDIA H100 GPU via NVlink C2C [14].



This work is licensed under a Creative Commons Attribution International 4.0 License.

HCDS '25, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1470-2/25/03

<https://doi.org/10.1145/3723851.3723853>

Table 1: NVLink versions. Data rate per link and direction.

Version	Data rate [GB/s]	# Lanes per Link	# Links	Architecture
1 [11]	20	8	4	Pascal
2 [12]	25	8	6	Volta
3 [13]	25	4	12	Ampere
4 [17] / C2C [14]	25	4	18	Hopper

2.1 NVLink Interconnect

NVIDIA’s NVLink interconnect combines multiple differential pairs (lanes) to a link [11]. A connection can use multiple links. An NVLink controller manages the protocol. Table 1 shows how the interconnect has evolved with each new GPU architecture. While NVLink 1 and 3 are GPU-to-GPU interconnects, CPUs with controllers for NVLink 2 and NVLink C2C exist. NVLink C2C combines a Grace CPU with a Hopper GPU in NVIDIA’s Grace-Hopper Superchip [14] and two Grace CPUs in their Grace Superchip [16]. NVLink C2C uses 18 links, providing a total bandwidth of 450 GB/s per direction [14, 16].

2.2 Anatomy of Memory Access

NVLink supports cache coherence and shared address spaces for CPU and GPU [1, 14]. The Grace CPU implements the AMBA CHI cache coherence protocol [9, 32], supported by NVLink C2C [20]. This makes both GPU and CPU memory byte-accessible to both processing units and allows the CPU to use the GPU memory as additional memory. When a CPU core reads data, it requests data from a program’s virtual address via a load instruction [8, 19]. This virtual address has to be translated into a physical address to access the data [8, 19, 26]. Existing translations are stored in a per-process page table, which is located in CPU memory [19]. A processor uses address translation hardware to receive these translations [8].

Translations are cached by a memory management unit (MMU) to avoid accessing memory for each translation [19, 33, 34]. The MMU consists of translation lookaside buffers (TLBs), which are separate caches for address translations, and a page table walk unit, used to fill the TLBs [8, 33, 34]. The MMU resides in each of the Grace CPU’s Neoverse V2 Core [30]. The MMU uses two fully associative L1 TLBs, separating instructions and data, and a four-way associative L2 TLB to cache address translations requested by the CPU core’s pipeline [33]. The MMU first looks up the TLBs for the requested translation [8, 19]. If the TLBs are missing the entry, the MMU will perform a page table walk to read the translation from memory and allocate new entries in the TLBs [8, 19, 33, 34]. It first accesses the data caches and then the page table in memory, in case of cache misses [8, 33].

Figure 1 visualizes the path from a CPU core to GPU memory for the NVIDIA GH200 chip. If the requested cache line is not cached or its cache line state is invalid, the core reads the cache line from the GPU memory. Each core has a private L1 and L2 cache, and a distributed last-level L3 cache [16]. If the cache line of a requested memory address is absent in the last-level cache, the cache line is requested from GPU memory via the NVLink interface.

The GPU’s NVLink interface is connected via a hub to the Crossbar (XBAR) interconnect. XBAR connects the hub, the graphics

processing clusters (GPCs), containing the GPU’s streaming multiprocessors, and the GPU’s L2 caches [11, 34, 36, 40]. The physical address request is routed to the GPU’s L2 cache. The L2 cache either holds the valid cache line or fetches it from the GPU memory. Once the cache line is received, it can be transferred via NVLink and loaded into the CPU’s cache hierarchy. Finally, the core’s L1 cache holds the valid requested cache line and fulfills the load instruction.

2.3 Benchmark Framework

For our experiments, we use CXL-Bench¹, a benchmark framework derived from Perma-Bench [7]. CXL-Bench allows measuring throughput and latency of memory accesses. It implements basic access patterns, namely sequential and random reads and writes. A user configures the memory region by setting its size and specifying the memory location as a non-uniform memory access (NUMA) node to which CXL-Bench binds the memory region. The user must assign threads to NUMA nodes or explicitly provide a set of cores to which CXL-Bench binds the threads. Benchmarks are duration-based, repeating the configured benchmark for a given time, or size-based, running until the benchmark threads have fully accessed a given memory region or until the threads performed a given number of operations. Alternative execution modes are custom operations and dependent reads used for latency measurements. Custom operations allow defining a sequence of memory accesses with the target memory region and access size for each access. For custom operations, a user configures up to two memory regions, allowing us to evaluate access patterns on different memory tiers, such as CPU and GPU memory. CXL-Bench supports running two benchmarks in parallel, allowing evaluation of scenarios where a system runs independent workloads that share resources.

CXL-Bench performs single instruction multiple data (SIMD) load and store instructions for its memory accesses and relies on compiler vector intrinsics (cf. [6]) rather than platform-specific SIMD intrinsics. A user-specified access size determines how many consecutive SIMD instructions are executed, depending on the size vector. E.g., an access size of 512 B and a vector size of 16 B results in 32 consecutive SIMD instructions.

3 Characterizing GPU Memory Access

We quantify CPU performance when accessing GPU memory. Section 3.1 introduces the NVIDIA GH200 evaluation system. We evaluate memory access throughput (Section 3.2), memory throughput expansion with data placed on both CPU and GPU memory (Section 3.3), and access latency (Section 3.4).

3.1 Setup

In our experiments, we use the QuantaGrid S74G-2U system, referred to as GH200. It utilizes an NVIDIA GH200 chip [14], which combines a Grace CPU [16] with a Hopper GPU (H100) [17] via NVLink C2C as an integrated chip. Figure 1 shows its components. The CPU’s memory region consists of 480 GB LPDDR5X integrated low-power memory that offers a bandwidth of up to 384 GB/s to the CPU [15, 18]. The GPU’s memory region provides 96 GB HBM3 memory with a 4000 GB/s bandwidth to the GPU’s memory controllers [14]. NVLink C2C connects CPU and GPU with a theoretical

¹CXL-Bench source code: <https://github.com/hpides/cxlbench/tree/HCDS-2025>

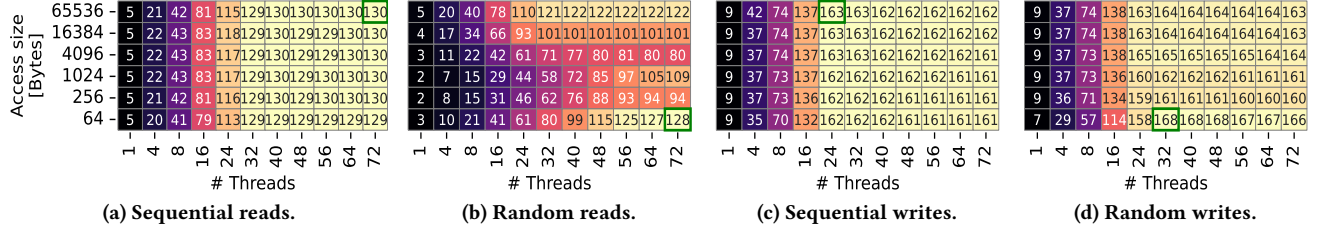


Figure 2: CPU throughput [GB/s] when accessing GPU memory with different access sizes and a fixed region of 7 GiB.

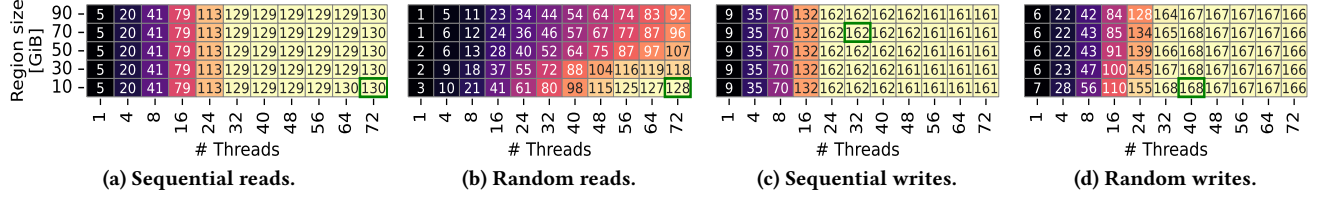


Figure 3: CPU throughput [GB/s] when accessing GPU memory with different region sizes and 64 B access sizes.

bandwidth of 450 GB/s per direction [14]. The NVIDIA Grace CPU implements 72 Neoverse V2 Cores [16] with a 64-B cache line size. Each core has a private 64 KB L1 and 1 MB L2 Cache [31]. The total L3 cache size is 114 MB [16]. NVIDIA Scalable Coherency Fabric is the CPU’s on-package interconnect that connects cores, the L3 cache, and system I/O [16]. The system exposes the CPU and GPU as NUMA nodes [14]. It runs Ubuntu 22.04 with the Linux kernel 6.2.0-1015-nvidia-64k with NVIDIA driver version 550.90.07. We perform all experiments with a 64 KiB page size.

CXL-Bench on ARM. We extend CXL-Bench (mainly tested on x86 systems before this work) to support ARM systems. The Grace CPU supports 128-bit SIMD instructions. On Neoverse V2, the compiler optimizes read instructions well to vectorized load instructions, loading 16 B via SIMD registers at a time. Reads result in the ld1 vector instruction. However, we observe that vector compiler intrinsics for write operations result in (byte-wise) scalar store instructions. We fix this by using the Neon intrinsic `vst1q_u8x42`, which uses four 16-B SIMD registers to store 64 B.

3.2 Throughput

We quantify the CPU’s throughput when accessing GPU memory via NVLink C2C. We use sequential and random access patterns for read and write workloads.

3.2.1 Setup. We measure average throughput for different access sizes and thread counts for sequential and random memory reads and writes. We allocate a 7 GiB memory region in the GPU memory. Sequential operations operate on continuous memory addresses. Random operations access randomly chosen addresses of the allocated memory address range. Each thread accesses a private chunk of the allocated memory region. We repeat accessing the memory region for 10 seconds and calculate the average throughput. We assign threads to physical cores. In addition, we perform the same experiment with a fixed access size of 64 B and different memory region sizes between 10 GiB and 90 GiB.

3.2.2 Results. In Figure 2a, we see that the maximum sequential read throughput to GPU memory is 130 GB/s. Thread counts over 28 do not show more gains in throughput. Thread counts between one and 20 achieve a throughput of about 5 GB/s to 6 GB/s per thread. With more threads, the throughput per thread is lower. The access size has marginal impact on sequential read throughput.

Random reads (see Figure 2b) achieve the highest throughput of 128 GB/s with 64 B accesses and 72 threads. Configurations with 52 or more threads and 64 B accesses reach throughputs of 122 GB/s and above. Access sizes of 16 KiB and 64 KiB do not show significant throughput gains with more than 28 threads. 4 KiB accesses show the lowest throughput of 80 GB/s when using all threads.

The maximum sequential write throughput (see Figure 2c) is 163 GB/s, using 24 to 32 threads for various access sizes. The throughput is similarly high when using more threads. Access sizes do not affect the throughput notably. From one to 16 threads, the throughput per thread is slightly under 9 GB/s. More threads start showing less gains in throughput.

The highest random write throughput (see Figure 2d) is 168 GB/s with a 64 B access size and 36 threads. Every access size reaches a throughput of 160 GB/s and higher when using 28 or more threads.

For comparison, accesses to CPU memory achieve a maximum sequential read throughput of 362 GB/s. Smaller access sizes reach lower throughput, e.g., 64 B accesses achieve a maximum of 316 GB/s. We measure a maximum of 345 GB/s with random reads. With sequential and random writes, we measure up to 370 GB/s.

Figure 3 shows the throughput for GPU memory with 64 B accesses and different memory region sizes. Region sizes have no or only a marginal impact on the throughput for all access types except random reads. For random reads, the throughput decreases with increasing region sizes.

3.2.3 Discussion. NVIDIA claims a unidirectional NVLink C2C bandwidth of up to 450 GB/s [14]. The highest we achieve is 168 GB/s with random writes. For each access pattern, memory

²Used Neon store intrinsic: https://developer.arm.com/architectures/instruction-sets/intrinsics/#q=vst1q_u8_x4

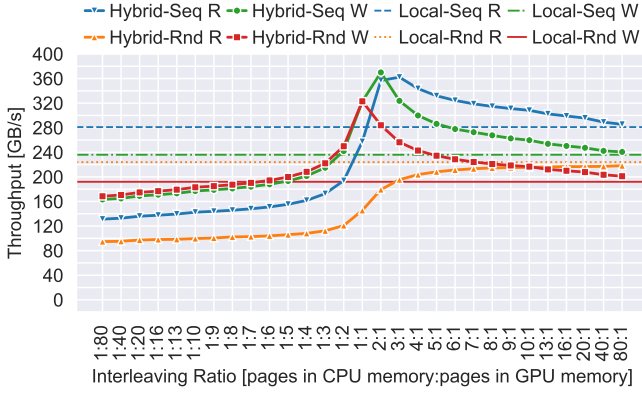


Figure 4: Throughput for sequential (Seq) and random (Rnd) reads (R) and writes (W) with pages interleaved across CPU and GPU memory (hybrid) and 72 threads. Horizontal lines show the throughput with data in CPU (local) memory.

accesses reach almost the highest throughput with 28 threads, regardless of the access size. Only random read throughput shows significant gains with more threads depending on the access size. Large access sizes for random reads profit from hardware prefetching. On CPU memory, large access sizes consistently outperform small sizes. With 130 GB/s, the maximum GPU memory read throughput is 36% of the highest CPU memory read throughput of 362 GB/s. With 168 GB/s, the maximum write throughput to GPU memory is 45% of the highest write throughput to CPU memory of 370 GB/s.

We measure the throughput of a GPU accessing CPU memory to see if we reach the maximum bandwidth for the interconnect. We use Nvbandwidth³, a benchmark tool provided by NVIDIA, to measure the throughput achieved by a GPU’s streaming multiprocessors and copy engine. We measure a maximum throughput of 350 GB/s for the Grace-Hopper, the expected throughput for this benchmark [15]. This throughput corresponds to 78% of the claimed unidirectional interconnect bandwidth of 450 GB/s. With that, the GPU accessing CPU memory reaches a much higher bandwidth than the CPU accessing GPU memory.

The access size marginally impacts write and sequential read throughput. Sequential accesses to consecutive memory addresses benefit from hardware prefetching. With random read accesses, the throughput depends more on the access size: larger access sizes increase the share of sequential SIMD accesses to consecutive memory addresses for each random access, benefiting from the hardware prefetcher.

3.3 Throughput Expansion

We evaluate whether accessing CPU and GPU memory increases the achievable throughput compared to accessing only CPU memory.

3.3.1 Setup. We measure the throughput for 64 B accesses to a 90 GiB memory region with 72 threads. We interleave the region’s pages across CPU and GPU memory with varying interleaving ratios $c:g$. The ratio defines a repetitive placement pattern in which c pages reside in CPU memory before the subsequent g pages reside in GPU memory. After allocating a memory region and pre-faulting

the pages, CXL-Bench places the pages via the `move_pages` system call. We perform sequential and random reads and writes. We repeat accessing the region for 10 seconds and report the average throughput. We measure the throughput for page interleaving ratios starting from 1:80 (with 98.8% of the pages in GPU memory) over 1:1 (round-robin) ratios, up to 80:1 (with 1.2% in GPU memory).

3.3.2 Results. Figure 4 shows the results. The horizontal lines serve as baselines. They show the access throughput achieved when all pages reside in CPU memory for the four access types. For each access type, the throughput achieved with interleaved pages when almost all pages (ratio 1:80) reside in GPU memory is lower than the baseline. The throughput grows with an increasing share of pages placed in CPU memory. For random writes, sequential writes, and sequential reads, the throughput exceeds the corresponding baseline throughput with the ratio 1:6, 1:2, and 2:1, respectively. These ratios translate to 86%, 67%, 33% of the pages residing in GPU memory. The throughput peaks at 323 GB/s with a 1:1 ratio for random writes, at 370 GB/s with 2:1 for sequential writes, and at 362 GB/s with 3:1 for sequential reads. With these ratios, 50%, 33.3%, and 25% of the pages reside in GPU memory. The performance peaks achieve speedup factors 1.68 $\times$, 1.57 $\times$, and 1.29 $\times$ relative to the baselines. Starting from a peak ratio, increasing the number of pages placed in GPU memory decreases the resulting throughput until it converges to the baseline performance. For random reads, no interleaving ratio results in a throughput exceeding the baseline. Instead, the throughput grows with an increasing share of pages in CPU memory until the throughput converges with the baseline.

3.4 Latency

We evaluate read latency of a CPU core when accessing GPU memory. We measure *idle latency* and *loaded latency*, i.e., the latency without and with load on the memory subsystem.

3.4.1 Setup. We measure the access latency with a single measurement thread by reading cache lines of a 1 GiB memory region located on the GPU card. We access all cache lines in this region and calculate their average access latency. We use a pointer-chasing workload. CXL-Bench randomizes the sequence of pointers when preparing the data region. Each cache line contains the address to the following cache line that needs to be loaded, which ensures that only one read is outstanding at a time [4] and reduces prefetcher effects. We refer to threads used to increase the load on the memory subsystem as *load threads*. We measure the system’s idle latency without any load threads. We execute multiple benchmark runs, incrementing the number of load threads to measure loaded latencies. We refer to the access type the load threads execute as *load operation*. We use CXL-Bench’s parallel benchmark feature to run our measurement thread and load threads in parallel. We measure loaded latencies for sequential and random reads and writes as load operations. We add threads that continuously run the load operation with cache line access sizes on a separate 7 GiB GPU memory region. We ensure that the latency measurement happens when all load threads are running. For comparison, we do the same experiment for CPU memory.

3.4.2 Results. We show the GPU and CPU memory access latencies per operation type in Figure 5. The first data point of a graph

³Nvbandwidth source repository: <https://github.com/NVIDIA/nvbandwidth>

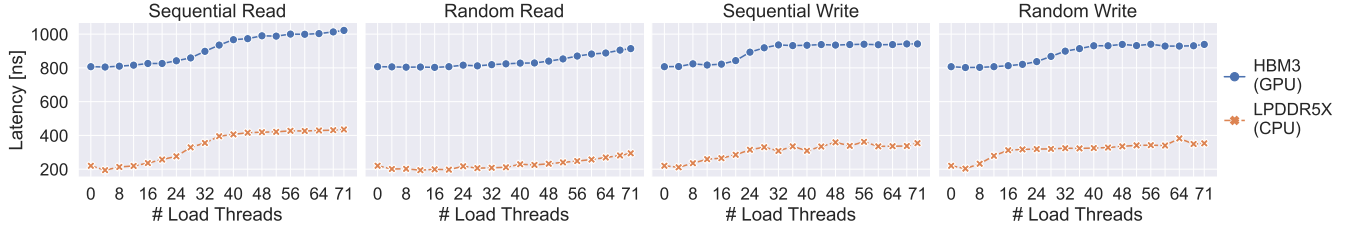


Figure 5: GPU memory access latencies with different load on the memory subsystem.

with no load threads corresponds to the idle latency. Other data points are loaded latencies. We measure an idle latency of 807 ns for GPU memory and 220 ns for CPU memory. The latencies for sequentially reading load threads increase continuously with added threads for both memory regions. Both lines have a similar trend, with the GPU memory latency being 500 ns to 600 ns higher. The highest measured latencies occur at 71 load threads with 1022 ns for GPU memory and 435 ns for CPU memory. Latencies measured with randomly reading load threads show a continuously increasing latency trend with about 600 ns difference between GPU and CPU memory accesses. Latencies are highest with 71 load threads, with accesses to GPU and CPU memory taking 914 ns and 294 ns, respectively. Latencies to GPU memory measured with sequentially writing load threads reach a maximum of 942 ns at 71 threads. The latency increases between 16 and 32 threads from 843 ns to 932 ns and stays flat in other areas. CPU memory access latency with sequentially writing load threads increases from 211 ns to 330 ns at four to 28 load threads. With randomly writing load threads, the latency stays between 802 ns and 821 ns for 20 load threads on GPU memory. Latency increases between 20 and 40 load threads to 931 ns and stays between 929 ns and 940 ns with additional threads. The CPU memory access latency increases between four and 16 threads from 203 ns to 312 ns. Latency increases slightly with each additional thread, reaching 353 ns with all threads.

3.4.3 Discussion. Memory accesses via NVLink C2C show high access latencies compared to CPU memory access. GPU memory access introduces a relatively constant additional latency of 500 ns to 600 ns compared to CPU memory access. Lam [29] measures a similar idle latency of 797 ns to GPU memory and 204 ns to CPU memory. He also measures an HBM3 latency of 300 ns from the GPU processors. Therefore, the HBM3 memory only introduces a minor portion of additional latency, and the interconnect causes most of the access latency to GPU memory.

We ensure that the measurement thread resolves one memory access at a time. In our throughput experiments, the CPU will prefetch when adjacent cache lines are accessed, leading to more cache hits. However, CPU cores are limited in the number of outstanding memory requests [1, 28]. An Arm Neoverse V2 core of the Grace CPU can queue up to 92 read and write requests [31, p. 76]. These queues can fill up for memory-intensive tasks like our throughput measurements when access latencies are too high.

In Section 3.2, we discuss that GPUs achieve higher throughput than CPUs. GPUs handle long-latency operations better, providing a higher degree of parallelism [27, p. 239:6, 239:8]. With CPUs' limitations of maximum outstanding memory requests and the relatively small number of cores, we assume that the CPU architectures

are not parallel enough to utilize the total bandwidth of the NVLink interconnect due to its high latency.

4 Database Operations On GPU Memory

We evaluate simulated access to a two-tier database index structure. The FPTree [38] is a database index structure distributed over DRAM and persistent memory. It places inner nodes in DRAM and leaf nodes in persistent memory. We model and evaluate operations on a hybrid tree-index structure using DRAM and GPU memory.

4.1 Setup

Benson et al. evaluate persistent memory with PerMA-Bench modeling index operations on an FPTree that uses DRAM and persistent memory [7]⁴. Similar to that experiment, we assume 1024-B leaf nodes, which hold 64 16-B records, and 2048-B inner nodes, which have 128 16-B records. We assume a tree with a billion entries and a height of six. The root, first, and second levels are assumed to reside in the L3 cache. We evaluate simulated lookup and update operations. We define custom operations with CXL-Bench for lookup and update operations on the FPTree. For a lookup, we must traverse five inner nodes and one leaf. Since we consider the first three levels as cached, we define a lookup as two 2048-B reads to the first memory tier, simulating reads of the last inner nodes. These are followed by a single 1024-B read to second-tier memory, simulating a read to a leaf node. An update consists of a lookup followed by three cache line-sized writes to the retrieved leaf for data, fingerprint, and slot index. The data is randomly positioned in the leaf, and the slot index and fingerprint are placed at the head of the leaf. We simulate writing to data by writing to the middle of the node with an offset of 512 B to the head.

We allocate a 5 GiB memory region in CPU memory and an 8 GiB region in GPU memory and perform 100 million operations. We run the benchmark with increasing thread numbers to observe the scalability. We benchmark each combination of the two memory types and report throughput in operations per second.

4.2 Results

Figure 6 shows the results for lookups. The achieved update throughput is very similar and omitted due to space limitations. Storing all nodes in CPU memory achieves the highest throughput of 42.85 million Ops/s and 42.17 million Ops/s for lookups and updates, using 16 threads. With inner nodes in CPU memory, the maximum throughput is 40.32 million Ops/s with 16 threads. The

⁴FPTree experiment configuration in PerMA-Bench: https://github.com/hpides/perma-bench/blob/7c8cc3a5cc0062475586fb95a9577fe8dc9cd71/workloads/operations/hybrid_tree_index.yaml

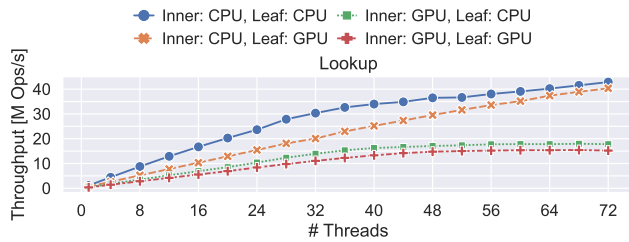


Figure 6: FPTree operations with different data placements.

speedup compared to all data in CPU memory is higher with lower thread counts and always reaches over 0.5 $\times$. With 40 threads or more, storing leaves in GPU memory achieves a speedup of 0.75 $\times$.

With inner nodes in GPU memory, the throughput trends are similar for both leaf nodes in CPU or GPU memory. For lookup operations, the trends increase to 48 threads and then flatten. For update operations, trends increase to 56 threads and then flatten. The throughput for both operations slightly drops with 72 threads. Storing leaves in CPU memory achieves the highest throughput of 17.95 million Ops/s with 68 threads on lookup operations. Update operations reach a throughput of 17.94 million Ops/s with 68 threads. Storing leaves in GPU memory achieves the highest throughput of 15.43 million Ops/s with 68 threads for lookup operations. Update operations reach 15.43 million Ops/s with 64 threads.

When using CPU and GPU memory, storing inner nodes in CPU memory and leaves in GPU memory performs best. The throughput scales well with threads and is slightly worse than storing all data in CPU memory. With all data in GPU memory, throughput still reaches a speedup of 0.3 $\times$ to 0.5 $\times$.

5 Discussion

The Grace-Hopper Chip shows promising results in all experiments. While 130 GB/s read throughput and 168 GB/s write throughput are far from the theoretical bandwidth of NVLink C2C, it achieves 37% to 48% of the Hopper GPU’s access throughput to CPU memory of 350 GB/s. For most system loads, the Grace CPU’s access latency to GPU memory is between 800 ns and 1000 ns. The high latencies indicate bottlenecks in the CPU architecture and limit throughput. The Grace CPU shows that high throughput is possible despite access to its high latencies to GPU memory. It offers a high level of parallelism, with many independent physical cores and an NVLink C2C interconnect with multiple links. Our hybrid tree-index experiment shows that GPU memory attached via NVLink C2C offers good performance and scalability, close to CPU memory, as a second memory tier.

PCIe 5 and 6 — the basis of CXL — offer 64 GB/s and 128 GB/s theoretical unidirectional bandwidth, respectively. This is lower than what NVLink C2C reaches with sustained throughput. In the context of database systems, we show that NVLink C2C-attached memory can achieve good performance, especially when used as secondary memory.

In terms of economic viability, we approximate HBM expenses by comparing two GPU models with similar specifications but different memory capacities. Based on the prices of NVIDIA A100 GPUs with 40 GB (~11 000 € [23]) and 80 GB (~22 250 € [24]) memory, we approximate 280 € per GB of HBM. As the A100 GPUs use

HBM2e memory, memory expansion via HBM3 is potentially more expensive. A setup with cheaper memory-only devices instead of GPUs with HBM is more economically viable for memory expansion: a DDR5-4800 16 GB DIMM costs about 44 € [25], resulting in less than 3 € per GB. The theoretical bandwidth of one DIMM with 4.8 GT/s is 38.4 GB/s. Assuming an NVLink C2C connection with 450 GB/s per direction (see Figure 1), 12 DIMMs are required on the memory expansion device to match the connection’s theoretical bandwidth. With a total cost of about 528 €, this memory setup would still be significantly cheaper than HBM on GPUs.

6 Related Work

Fusco et al. [22] evaluate the memory access performance of both CPU and GPU on the GH200. Their work includes memory accesses from CPU to GPU memory via scalar instructions. IBM [1] explains how their Power9 CPU integrates NVLink 2 to connect NVIDIA Volta GPUs. Their work includes benchmarking the read and write throughput of NVLink 2 when the CPU accesses GPU memory.

Lutz et al. [35] use the IBM Power System AC922 to show that NVLink 2 can overcome transfer bottlenecks and is suitable for non-partitioning hash joins. Their work includes analyzing NVLink 2’s features, comparing transfer strategies between CPU and GPU, and using it to accelerate database operations. Lutz et al. [36] further propose Triton Join. Their work includes a microbenchmark to measure the throughput of different transfer methods between CPU and GPU on the AC922, including CPU access to GPU memory.

7 Conclusion

We evaluate NVLink C2C from a memory disaggregation perspective, focusing on CPU memory access to the device memory. We investigate whether NVLink C2C is suitable for memory expansion. We measure the throughput of sequential and random reads and writes for the Grace CPU accessing Hopper GPU memory. We compare the results with access throughput from the CPU to CPU memory and from the GPU to CPU memory. While CPU accesses to GPU memory cannot saturate the interconnect, the access throughput achieves 37% to 48% of the Hopper GPU’s access throughput to CPU memory. Our latency measurements provide latencies for the CPU’s access to CPU and GPU memory, with different levels of load on the memory subsystem. We generally measure high latencies for GPU memory access. As GPUs achieve higher throughput over the interconnects, CPUs are limited in their degree of parallelism and cannot fully utilize the interconnect. We put the memory access via NVLink C2C into the context of database systems by modeling accesses to a hybrid tree-index structure. We measure tiered configurations with GPU memory and compare them to other configurations. We find that configurations with GPU memory reach performance close to the CPU memory measurements. Our experiments show that memory access latency from the CPU to GPU memory is high, with between about 800 ns to 1000 ns. We measure an access throughput of up to 168 GB/s.

Acknowledgments

This work was partially funded by the German Research Foundation (ref. 414984028), the European Union’s Horizon 2020 research and innovation programme (ref. 957407), and by SAP.

References

- [1] 2018. Functionality and performance of NVLink with IBM POWER9 processors. *IBM J. Res. Dev.* 62, 4/5 (2018), 9:1–9:10. <https://doi.org/10.1147/JRD.2018.2846978>
- [2] Inc. Advanced Micro Devices. 2023. AMD CDNA 3 Architecture. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf> Whitepaper.
- [3] Marcos K. Aguilera, Emmanuel Amaro, Nadav Amit, Erika Hunhoff, Anil Yelam, and Gerd Zellweger. 2023. Memory disaggregation: why now and what are the challenges. *ACM SIGOPS Oper. Syst. Rev.* 57, 1 (2023), 38–46. <https://doi.org/10.1145/3606557.3606563>
- [4] Alif Ahmed and Kevin Skadron. 2019. Hopscotch: a micro-benchmark suite for memory performance evaluation. In *Proceedings of the International Symposium on Memory Systems, MEMSYS 2019, Washington, DC, USA, September 30 - October 03, 2019*. ACM, 167–172. <https://doi.org/10.1145/3357526.3357574>
- [5] Christoph Anneser, Lukas Vogel, Ferdinand Gruber, Maximilian Bandle, and Jana Giceva. 2023. Programming Fully Disaggregated Systems. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems, HOTOS 2023, Providence, RI, USA, June 22-24, 2023*, Malte Schwarzkopf, Andrew Baumann, and Natacha Crooks (Eds.). ACM, 188–195. <https://doi.org/10.1145/3593856.3595889>
- [6] Lawrence Benson, Richard Ebeling, and Tilmann Rabl. 2023. Evaluating SIMD Compiler-Intrinsics for Database Systems. In *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023 (CEUR Workshop Proceedings, Vol. 3462)*, Rajesh Bordawekar, Cinzia Cappiello, Vasilis Efthymiou, Lisa Ehrlinger, Vijay Gadepally, Sainyam Galhotra, Sandra Geisler, Sven Gropp, Le Gruenwald, Alon Y. Halevy, Hazar Harmouch, Oktie Hassanzadeh, Ihab F. Ilyas, Ernesto Jiménez-Ruiz, Sanjay Krishnan, Tirthankar Lahiri, Guoliang Li, Jiaheng Lu, Wolfgang Mauere, Umar Farooq Minhas, Felix Naumann, M. Tamer Özsu, El Kindi Rezig, Kavitha Srinivas, Michael Stonebraker, Satyanarayana R. Valluri, Maria-Esther Vidal, Haixun Wang, Jiannan Wang, Yingjun Wu, Xun Xue, Mohamed Zait, and Kai Zeng (Eds.). CEUR-WS.org, Vol. 3462/ADMS5.pdf
- [7] Lawrence Benson, Leon Papke, and Tilmann Rabl. 2022. PerMA-Bench: Benchmarking Persistent Memory Access. *Proc. VLDB Endow.* 15, 11 (2022), 2463–2476. <https://doi.org/10.14778/3551793.3551807>
- [8] Abhishek Bhattacharjee. 2017. Appendix L: Advanced Concepts on Address Translation. (2017). In *Computer architecture — A quantitative approach* (6th ed.), John L. Hennessy and David A. Patterson (Eds.). Morgan Kaufmann, Cambridge, MA, USA, Appendix L, 1–69.
- [9] Magnus Bruce. 2023. Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for Next-Generation Cloud Computing, ML and HPC Workloads. In *Hot Chips 2023 Presentation*. <https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC2023.Arm.MagnusBruce.v04.FINAL.pdf> Accessed: 2025-01-20.
- [10] CXL Consortium. [n. d.]. Compute Express Link specification, Revision 3.0. CXL. <https://www.computeexpresslink.org/> Accessed: 2025-02-07.
- [11] NVIDIA Corporation. 2016. NVIDIA Tesla P100 - The Most Advanced Datacenter Accelerator Ever Built. <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf> Whitepaper V1.1.
- [12] NVIDIA Corporation. 2017. NVIDIA TESLA V100 GPU ARCHITECTURE - THE WORLD'S MOST ADVANCED DATA CENTER GPU. <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf> Whitepaper V1.1.
- [13] NVIDIA Corporation. 2020. NVIDIA A100 Tensor Core GPU Architecture - UNPRECEDENTED ACCELERATION AT EVERY SCALE. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf> Whitepaper V1.0.
- [14] NVIDIA Corporation. 2023. NVIDIA GH200 Grace Hopper Superchip Architecture - Performance and Productivity for Strong-Scaling HPC and Giant AI Workloads. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper-whitepaper> V1.12.
- [15] NVIDIA Corporation. 2023. NVIDIA GH200 Grace Hopper Superchip Benchmark Step-by-Step Guide. <https://docs.nvidia.com/gh200-superchip-benchmark-guide.pdf>
- [16] NVIDIA Corporation. 2023. NVIDIA Grace CPU Superchip Whitepaper - Performance and Efficiency for the Modern Data Center. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-cpu-superchip-whitepaper> V1.0.
- [17] NVIDIA Corporation. 2023. NVIDIA H100 Tensor Core GPU Architecture - EXCEPTIONAL PERFORMANCE, SCALABILITY, AND SECURITY FOR THE DATA CENTER. <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper-whitepaper> V1.04..
- [18] NVIDIA Corporation. 2025. Grace Performance Tuning Guide. <https://docs.nvidia.com/grace-perf-tuning-guide/index.html> Accessed: 2025-03-30.
- [19] Ulrich Drepper. 2007. What Every Programmer Should Know About Memory. (01 2007). <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- [20] Cliff Edwards. 2022. NVIDIA Opens NVLink for Custom Silicon Integration. <https://nvidianews.nvidia.com/news/nvidia-opens-nvlink-for-custom-silicon-integration>
- [21] Mohammad Ewais and Paul Chow. 2023. Disaggregated Memory in the Data-center: A Survey. *IEEE Access* 11 (2023), 20688–20712. <https://doi.org/10.1109/ACCESS.2023.3250407>
- [22] Luigi Fusco, Mikhail Khalilov, Marcin Chrapek, Giridhar Chukkappalli, Thomas Schulthess, and Torsten Hoefer. 2024. Understanding data movement in tightly coupled heterogeneous systems: A case study with the Grace Hopper superchip. *arXiv preprint arXiv:2408.11556* (2024).
- [23] Geizhals. 2025. NVIDIA A100 40 GB price as of 2025-02-06. <https://geizhals.eu/fujitsu-s26361-f3396-14-a1774093.html> Accessed: 2025-02-07.
- [24] Geizhals. 2025. NVIDIA A100 80 GB price as of 2025-01-27. <https://geizhals.eu/fujitsu-s26361-f3935-1616-a1762880.html> Accessed: 2025-02-07.
- [25] Geizhals. 2025. Samsung DIMM 16GB, DDR5-4800, price as of 2025-02-06. <https://geizhals.de/samsung-dimm-ddr5-v93906.html> Accessed: 2025-02-07.
- [26] John L. Hennessy and David A. Patterson. 2017. Appendix L: Advanced Concepts on Address Translation. (2017). In *Computer architecture — A quantitative approach* (6th ed.), John L. Hennessy and David A. Patterson (Eds.). Morgan Kaufmann, Cambridge, MA, USA, Appendix B, B1–B60..
- [27] Pieter Hijma, Stijn Heldens, Alessio Sclocco, Ben van Werkhoven, and Henri E. Bal. 2023. Optimization Techniques for GPU Programming. *ACM Comput. Surv.* 55, 11 (2023), 239:1–239:81. <https://doi.org/10.1145/3570638>
- [28] Roland Kühn, Jan Mühlig, and Jens Teubner. 2024. How to Be Fast and Not Furious: Looking Under the Hood of CPU Cache Prefetching. In *Proceedings of the 20th International Workshop on Data Management on New Hardware, DaMoN 2024, Santiago, Chile, 10 June 2024*, Carsten Binnig and Nesime Tatbul (Eds.). ACM, 9:1–9:10. <https://doi.org/10.1145/3662010.3663451>
- [29] Chester Lam. 2024. Grace Hopper, Nvidia's Halfway APU. <https://chipsandcheese.com/2024/07/31/grace-hopper-nvidias-halfway-apu/> Accessed: 2024-08-28.
- [30] Arm Limited. 2022. AMBA 5 CHI Architecture Specification. <https://developer.arm.com/documentation/ih0050/latest> Version F.
- [31] Arm Limited. 2022. Arm Neoverse V2 Core Technical Reference Manual. <https://developer.arm.com/documentation/102375/0002/>
- [32] Arm Limited. 2024. AMBA CHI Architecture Specification. <https://developer.arm.com/documentation/ih0050/g/> Version G.
- [33] Arm Limited. 2024. Arm Neoverse V2 - PMU Guide. <https://developer.arm.com/documentation/109709/r0p2> Revision: r0p2.
- [34] Clemens Lutz. 2022. Scalable data management using GPUs with fast interconnects. Ph.D. Dissertation. Technical University of Berlin, Germany. <https://nbn-resolving.org/urn:nbn:de:101:1-2022122101021675299107>
- [35] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2020. Pump Up the Volume: Processing Large Data on GPUs with Fast Interconnects. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 1633–1649. <https://doi.org/10.1145/3318464.3389705>
- [36] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2022. Triton Join: Efficiently Scaling to a Large Join State on GPUs with Fast Interconnects. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 1017–1032. <https://doi.org/10.1145/3514221.3517911>
- [37] Tobias Maltenberger, Ivan Ilic, Ilin Tolovski, and Tilmann Rabl. 2022. Evaluating Multi-GPU Sorting with Modern Interconnects. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 1795–1809. <https://doi.org/10.1145/3514221.3517842>
- [38] Ismail Oukid, Johan Lasperas, Anisoara Nica, Thomas Willhalm, and Wolfgang Lehner. 2016. FPTree: A Hybrid SCM-DRAM Persistent and Concurrent B-Tree for Storage Class Memory. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). ACM, 371–386. <https://doi.org/10.1145/2882903.2915251>
- [39] Ismail Oukid and Lucas Lersch. 2018. On the Diversity of Memory and Storage Technologies. *Datenbank-Spektrum* 18, 2 (2018), 121–127. <https://doi.org/10.1007/S13222-018-0287-8>
- [40] Robert Sohigian. 21. NVSwitch Accelerates NVIDIA DGX-2. <https://developer.nvidia.com/blog/nvswitch-accelerates-nvidia-dgx2/> Accessed: 2025-02-03.
- [41] Dimitrios Ziakas, Allen Baum, Robert A. Maddox, and Robert J. Safranek. 2010. Intel® QuickPath Interconnect Architectural Features Supporting Scalable System Architectures. In *IEEE 18th Annual Symposium on High Performance Interconnects, HOTI 2010, Google Campus, Mountain View, California, USA, August 18-20, 2010*, Fabrizio Petrini, Dennis Abts, Ron Brightwell, Pavan Balaji, and Cyriel Minkenberg (Eds.). IEEE Computer Society, 1–6. <https://doi.org/10.1109/HOTI.2010.24>